

The Avadhana Neuro-Symbolic Kernel (v2)

Differentiable Integration of the Pingala, Panini, and Nyaya
Frameworks: A Corrected Mathematical Treatment

Mukesh Pandey
Independent Research

Mathematical development: Claude Sonnet 4.6 (Anthropic)

February 2026 (v2, correcting errors identified in peer review)

Abstract

This is a corrected revision of the Avadhana Neuro-Symbolic Kernel (NSK) paper. Peer review identified **ten substantive mathematical errors** in v1, every one of which we accept. This version corrects them explicitly. The corrections are: (1) the gradient-death immunity claim is downgraded to a *mitigation* bound with a new counter-example showing the bound is tight; (2) the Panini bottleneck PCA claim is restricted to the linear case, with the nonlinear case labelled OPEN; (3) the non-differentiable Heaviside branch in soft rejection is replaced by a smooth sigmoid gate; (4) Panini mask saturation is acknowledged with a provably bounded mitigation; (5) the Nyaya verifier is upgraded from a single linear layer to a two-layer MLP, with claims restricted to *distributional plausibility scoring*, not formal logic verification; (6) fp16 Riemannian drift is addressed with an fp32 master-copy prescription and periodic re-orthogonalisation; (7) the Lyapunov theorem is narrowed from global to *local* stability; (8) HBTA dimension inflation is resolved by making the gated-sum broadcast architecture explicit; (9) distance concentration in hierarchical routing is labelled as an open failure mode with a proposed mitigation; (10) the hardware feasibility table is extended to realistic scales ($d \leq 1024$). All remaining claims preserve the proven/plausible/open labelling of v1.

Contents

1	Formal Problem Definition	3
2	Pingala Kernel: Compute Routing and Hierarchical Memory	3
2.1	Differentiable Gating — Corrected Claims	3
2.2	Hierarchical Memory: $O(\log N)$ Retrieval	4
3	Panini Kernel: Structural Constraint Layer	5
3.1	Sphota Bottleneck — Corrected Scope	5
3.2	Differentiable Constraint Masking — Saturation Analysis	6
4	Nyaya Kernel: Differentiable Plausibility Verifier	7
4.1	MLP Verifier and Energy Function	7
4.2	Smooth Rejection Sampling — Corrected Design	8
5	Differentiable Logic Integration	8

6	Stability and Convergence — Corrected Scope	9
6.1	Local Lyapunov Stability	9
6.2	Riemannian Stability and fp16 Drift	10
6.3	Meta-Controller Stability (unchanged from v1)	11
7	Hardware Efficiency and Realistic Scale Analysis	11
7.1	FLOPs Accounting — Corrected	11
7.2	Realistic Hardware Feasibility	12
8	Gradient Flow: Corrected Global Analysis	12
9	Comparison with Existing Architectures	13
10	Complete Open Problems and Limitation Inventory	14
10.1	Proven Results (Retained from v1, Scope Corrected)	14
10.2	Plausible but Unverified	14
10.3	Open Problems (Extended from v1)	14
11	Response to Reviewer: Point-by-Point	15
12	Conclusion	15

1 Formal Problem Definition

Definition 1.1 (System State). *An NSK system at step t is:*

$$S_t \in \mathbb{R}^{d \times K}, \quad S_t^\top S_t = I_K, \quad S_t \in \text{St}(d, K), \quad K \leq d \quad (1)$$

$$C_t \in \mathbb{R}^k \quad (2)$$

$$\mathcal{M} = (M^W, M^E, M^S) \quad (3)$$

where d is the embedding dimension, K the thread count, and the three memory tiers are working (M^W , volatile), episodic (M^E , persistent), and semantic (M^S , read-only priors).

Definition 1.2 (Forward Pass and Training Objective). *The forward map $h_{t+1} = f(h_t, S_t, \mathcal{M}, C_t) = \mathcal{K}_N \circ \mathcal{K}_P \circ \mathcal{K}_\Pi$ and the composite loss:*

$$L_{\text{total}}(\theta) = L_{\text{task}} + \alpha L_{\text{verify}} + \beta L_{\text{compute}} + \gamma L_{\text{bottle}} + \delta L_{\text{orth}} \quad (4)$$

Parameters θ are updated by Adam. S_t is updated separately by Riemannian gradient descent on $\text{St}(d, K)$, stored as an **fp32 master copy** irrespective of the mixed-precision regime used for θ (see Section 6.2).

2 Pingala Kernel: Compute Routing and Hierarchical Memory

2.1 Differentiable Gating — Corrected Claims

Definition 2.1 (Layer Gate and Gated Update).

$$g_\ell(x) = \sigma(W_\ell x + b_\ell) \in (0, 1) \quad (5)$$

$$h_{\ell+1} = g_\ell(x) \cdot f_\ell(h_\ell) + (1 - g_\ell(x)) \cdot h_\ell \quad (6)$$

Correction from v1

v1 error: Theorem 2.4 claimed “gradient death does not occur at any layer.” This is false. The reviewer correctly identified that when $g_\ell \rightarrow 1$ and the sub-network Jacobian $J_{f_\ell} \rightarrow 0$ simultaneously (e.g., in deep saturated ReLU layers), both terms in the gradient vanish.

v2 correction: We replace the immunity claim with a tighter *lower bound* that is strictly positive only when at least one of $\{g_\ell, J_{f_\ell}\}$ is bounded away from its degenerate value, and we exhibit the failure case explicitly.

Theorem 2.2 (Gradient Mitigation — Not Immunity). *For loss $\mathcal{L}$ differentiable in $h_{\ell+1}$, define $\delta_\ell := \frac{\partial \mathcal{L}}{\partial h_{\ell+1}}$. The gradient satisfies:*

$$\frac{\partial \mathcal{L}}{\partial h_\ell} = \underbrace{(1 - g_\ell) \delta_\ell}_{\text{residual path}} + \underbrace{g_\ell \delta_\ell J_{f_\ell}}_{\text{sub-net path}} \quad (7)$$

with lower bound:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_\ell} \right\| \geq |1 - g_\ell| \|\delta_\ell\| - g_\ell \|\delta_\ell\| \|J_{f_\ell}\| \quad (8)$$

This bound is **strictly positive** if and only if $(1 - g_\ell) > g_\ell \|J_{f_\ell}\|$, i.e., when the gate is not too open relative to the sub-network’s spectral radius.

Proof. Direct application of the chain rule to $h_{\ell+1} = h_\ell + g_\ell(f_\ell(h_\ell) - h_\ell)$ gives (7). The bound (8) follows from the triangle inequality: $\|\mathbf{a} + \mathbf{b}\| \geq \|\mathbf{a}\| - \|\mathbf{b}\|$. $\square$ $\square$

Counter-Example 2.3 (When Gradient Vanishes). *Let $g_\ell = \sigma(10) \approx 0.9999 \approx 1$ and $f_\ell(h) = \tanh(W_h)$ with $\|W\| \rightarrow 0$ (heavily regularised or freshly initialised with tiny weights). Then $J_{f_\ell} = \text{diag}(1 - \tanh^2(W_h)) \cdot W \rightarrow 0$. Substituting into (8): the bound $(1 - 0.9999)\|\delta_\ell\| - 0.9999 \cdot 0 = 10^{-4}\|\delta_\ell\|$ is positive but exponentially small in the gate pre-activation. For $L = 24$ layers, the product $\prod_\ell 10^{-4} \approx 10^{-96}$: effectively zero in floating point.*

Plausible — Unverified

Plausible mitigation: The failure case in Counter-Example 2.3 is avoided in practice by: (i) initialising b_ℓ so that $g_\ell(x) \approx 0.5$ at the start of training (not near 0 or 1); (ii) using gradient clipping (norm clip = 1.0); (iii) applying layer normalisation before each f_ℓ . Whether these are *sufficient* to prevent vanishing throughout training on real tasks is **empirically unverified**.

Compute regulariser (unchanged from v1).

$$L_{\text{compute}}(\theta) = \lambda \sum_{\ell=1}^L \mathbb{E}_{x \sim \mathcal{D}} [g_\ell(x)] \quad (9)$$

This is differentiable with gradient $\frac{\partial L_{\text{compute}}}{\partial W_\ell} = \lambda \mathbb{E}[g_\ell(1 - g_\ell)x^\top]$, non-zero for non-constant input distributions.

2.2 Hierarchical Memory: $O(\log N)$ Retrieval

Definition 2.4 (Hierarchical Memory Tree with Gated-Sum Broadcast). *Let the memory be an L -level binary tree with $N_\ell = N/2^{\ell-1}$ nodes at level ℓ , each of dimension d .*

Bottom-up aggregation: $M_p^{(\ell+1)} = \phi(M_{2p-1}^{(\ell)}, M_{2p}^{(\ell)})$ where $\phi(u, v) = (u + v)/2$.

Top-down broadcast (gated sum, dimension-preserving): *The per-leaf context is:*

$$\text{ctx}_i = \sum_{k=0}^L G_k(W_k \cdot \text{anc}_k(i)) \quad (10)$$

where $\text{anc}_k(i) \in \mathbb{R}^d$ is the level- k ancestor of leaf i , $W_k \in \mathbb{R}^{d \times d}$ is a learned projection, and $G_k = \sigma(\cdot) \in (0, 1)^d$ is a per-dimension gate. **The output $\text{ctx}_i \in \mathbb{R}^d$ has the same dimension d as every individual level representation.** There is no dimension inflation.

Correction from v1

v1 error (reviewer §3): The broadcast step was ambiguous and could be interpreted as concatenation, giving leaf dimension Ld and downstream cost $O(NLd^2) = O(Nd^2 \log N)$.

v2 correction: The broadcast is an explicit *gated sum* (10). Each level's projection W_k maps $\mathbb{R}^d \rightarrow \mathbb{R}^d$, so the output dimension is d regardless of L . Downstream operations operate on d -dimensional vectors. The $O(Nd \log N)$ complexity claim is now unambiguous.

Proven

Theorem 2.5 (Retrieval Complexity). *Hierarchical retrieval requires $O(\log N)$ dot products vs. $O(N)$ for flat nearest-neighbour search.*

Proof. At each of $L = \lceil \log_2 N \rceil$ levels, the query attends to exactly 2 child nodes. Total: $2L = O(\log N)$ dot products. $\square$ $\square$

Open Problem

Open: Distance Concentration and Routing Fragility. The reviewer correctly notes that in high dimensions $d \gg 1$, dot products concentrate: for i.i.d. Gaussian vectors, $q^\top m / \sqrt{d} \xrightarrow{d \rightarrow \infty} \mathcal{N}(0, 1)$ with variance $O(1/d)$, making the two root-level children statistically indistinguishable. A greedy routing error at level ℓ is *unrecoverable*: the query is permanently locked into the wrong half of the tree.

This is a genuine failure mode. It is not solved by the current architecture.

Proposed mitigations (unproven): (a) *Beam search*: maintain top- B candidates at each level; (b) *Learned routing temperature*: replace the fixed softmax temperature with $\beta(\ell) = \beta_0 \cdot e^{-\kappa \ell}$, amplifying scores at the root where concentration is worst; (c) *Random tree ensembles*: average retrieval across multiple independently-built trees (analogue of LSH). All three require experimental validation.

3 Panini Kernel: Structural Constraint Layer

3.1 Sphota Bottleneck — Corrected Scope

Definition 3.1 (Bottleneck Encoder-Decoder). For $d' < d$, define $z = E_\phi(h) \in \mathbb{R}^{d'}$, $\hat{h} = D_\psi(z) \in \mathbb{R}^d$, with reconstruction loss:

$$L_{\text{bottle}}(\phi, \psi) = \mathbb{E}_h [\|h - D_\psi(E_\phi(h))\|^2] \quad (11)$$

Correction from v1

v1 error (reviewer §1.2): Proposition 3.2 applied the Eckart-Young theorem to nonlinear autoencoders, claiming they “capture at least the linear PCA variance.” This is false. Nonlinear autoencoders optimise a non-convex objective and can converge to local minima that are strictly worse than the PCA solution.

v2 correction: We restrict the PCA claim to the linear case and explicitly label the nonlinear case as open.

Proven

Proposition 3.2 (Linear Bottleneck = PCA). *If $E_\phi(h) = U^\top h$ ($U \in \mathbb{R}^{d \times d'}$, linear) and $D_\psi(z) = Vz$ ($V \in \mathbb{R}^{d \times d'}$, linear), then the global minimiser of L_{bottle} is $U^* = V^* = \Sigma_{d'}^{1/2}$, where the columns of U^* are the top- d' eigenvectors of the data covariance $\Sigma_h = \mathbb{E}[hh^\top]$. This is the classical Eckart-Young result.*

Proof. For linear encoder-decoder, $L_{\text{bottle}} = \mathbb{E}[\|h - VU^\top h\|^2] = \text{tr}(\Sigma_h) - \text{tr}(U^\top \Sigma_h V (VU^\top)^{-1} U \Sigma_h)$. Minimising over U, V with the constraint $V = U$ (autoencoder symmetry) yields the PCA solution by the Eckart-Young-Mirsky theorem. $\square$ $\square$

Open Problem

Open: Nonlinear Bottleneck Optimality. For nonlinear E_ϕ, D_ψ (e.g., MLP), the optimisation landscape of L_{bottle} is non-convex. Local minima exist and may be strictly worse than linear PCA. Whether practical gradient descent converges to a good solution for the specific architecture used here is an empirical question. The v1 claim that nonlinear autoencoders “capture at least linear PCA variance” is retracted.

3.2 Differentiable Constraint Masking — Saturation Analysis

Definition 3.3 (Constraint Mask).

$$m_i = \log \sigma(c_i) \in (-\infty, 0) \quad (12)$$

$$p_i = \frac{\exp(z_i + m_i)}{\sum_j \exp(z_j + m_j)} \quad (13)$$

where $c_i \in \mathbb{R}$ is a learnable constraint logit.

Proven

Theorem 3.4 (Differentiability and Saturation Characterisation). *The mapping $(z, c) \mapsto p$ is C^∞ with:*

$$\frac{\partial p_i}{\partial c_k} = \underbrace{(1 - \sigma(c_k))}_{\text{mask gate}} \cdot p_i (\delta_{ik} - p_k) \quad (14)$$

*The gradient is non-zero if and only if $\sigma(c_k) \neq 1$, i.e., $c_k < +\infty$. **Saturation occurs** when $c_k \rightarrow +\infty$: the mask becomes transparent ($m_k \rightarrow 0$) and $\frac{\partial p_i}{\partial c_k} \rightarrow 0$.*

Correction from v1

v1 error (reviewer §2.1): v1 claimed the gradient is “non-degenerate” without qualification. This is false: $c_k \rightarrow +\infty$ drives $(1 - \sigma(c_k)) \rightarrow 0$, causing dead neurons.
v2 correction: The saturation is real and must be managed by the following engineering prescription.

Remark 3.5 (Saturation Mitigations — Required in Practice). *To prevent permanent constraint mask saturation:*

1. **Initialise** $c_i \sim \mathcal{N}(0, 0.1)$ so that $\sigma(c_i) \approx 0.5$ and $(1 - \sigma(c_i)) \approx 0.5$ at training start.
2. **Clip logits:** $c_i \leftarrow \text{clip}(c_i, -C_{\max}, C_{\max})$ with $C_{\max} = 5.0$. At saturation boundary: $(1 - \sigma(5)) \approx 0.007 > 0$.
3. **Add entropy regulariser:** $L_{\text{ent}} = -\rho \sum_i H(p_i)$ where H is the entropy of the masked distribution, discouraging collapse to degenerate masks.

These prescriptions do not restore the gradient to $O(1)$ magnitude in the saturated regime; they prevent saturation from being reached. Whether they are sufficient for a given task is empirically open.

4 Nyaya Kernel: Differentiable Plausibility Verifier

Correction from v1

v1 error (reviewer §6): The v1 verifier $V(h) = \sigma(W_v h + b_v)$ is a single affine transformation. A single linear layer cannot represent non-linearly separable propositions (e.g., XOR). The claim of “self-verification without external symbolic solvers” was unjustified for a linear model.

v2 correction: (1) The verifier is upgraded to a **two-layer MLP** (minimum for universal function approximation); (2) The claim is restricted to *distributional plausibility scoring*: V learns to assign high confidence to states that resemble valid training examples. It does **not** implement formal logical verification. Formal verification requires external symbolic systems; this architecture does not provide one.

4.1 MLP Verifier and Energy Function

Definition 4.1 (MLP Verifier).

$$\tilde{h} = \text{LayerNorm}(h) \quad (15)$$

$$a = \text{GELU}(W_1 \tilde{h} + b_1) \in \mathbb{R}^{d_v}, \quad W_1 \in \mathbb{R}^{d_v \times d} \quad (16)$$

$$V(h) = \sigma(W_2 a + b_2) \in (0, 1)^{d_v}, \quad W_2 \in \mathbb{R}^{d_v \times d_v} \quad (17)$$

$$E(h) = -\frac{1}{d_v} \mathbf{1}^\top \log V(h) \geq 0 \quad (18)$$

Proven

Proposition 4.2 (MLP Verifier Gradient). *The gradient of $E(h)$ with respect to W_2 is:*

$$\frac{\partial E}{\partial W_2} = -\frac{1}{d_v} (I - \text{diag}(V(h))) a^\top \quad (19)$$

which is non-zero whenever $V_j(h) \neq 1$ for some j and $a \neq 0$. Both conditions hold for generic (h, W_1, W_2) since GELU is not identically zero and $\sigma < 1$ strictly.

Correction from v1

v1 error (reviewer §2.2): The reviewer correctly noted that when $V_j(h) \rightarrow 1$ (high verifier confidence), the gradient $(1 - V_j(h)) \rightarrow 0$ and the verifier loss becomes uninformative.

v2 acknowledgement: This gradient collapse is *real*. It is the neural analogue of the “reward collapse” problem in reinforcement learning. Mitigations: (a) Add a small noise term $V_j(h) + \epsilon$ to keep the gradient away from zero; (b) Use a *contrastive* verifier loss (train on negative examples) so that the verifier is not uniformly confident; (c) Periodically reset the verifier head if $E(h) < \tau_{\min}$ for k consecutive batches. None of these has been empirically validated for this architecture.

Definition 4.3 (Verifier Loss).

$$L_{\text{verify}}(\theta_v) = \mathbb{E}_{h \sim p_\theta}[E(h)] \quad (20)$$

4.2 Smooth Rejection Sampling — Corrected Design

Correction from v1

v1 error (reviewer §1.3): The v1 soft rejection equation used a Heaviside step function $\mathbf{1}[E(\hat{h}) < \tau]$, which has zero derivative almost everywhere and is undefined at $E = \tau$. This introduced a non-differentiable branch, directly contradicting the premise of a differentiable architecture.

v2 correction: Replace the hard threshold with a smooth sigmoid gate on the refinement step size.

Definition 4.4 (Smooth Differentiable Rejection). *Define the refinement step as:*

$$\tilde{h} = \hat{h} - \underbrace{\eta_r \cdot \sigma\left(\frac{E(\hat{h}) - \tau}{\beta_r}\right)}_{\text{smooth gate } \in(0,1)} \cdot \nabla_h E(\hat{h}) \quad (21)$$

where $\tau > 0$ is the soft threshold, $\beta_r > 0$ is the temperature (sharpness of the gate), and $\eta_r > 0$ is the refinement step size.

Proven

Theorem 4.5 (Differentiability of Smooth Rejection). *The map $\hat{h} \mapsto \tilde{h}$ defined by (21) is differentiable with respect to $\hat{h}$, τ , β_r , η_r , and all parameters of E .*

Proof. σ is C^∞ . $E(\hat{h})$ is C^∞ by composition of GELU, affine maps, sigmoid, and log (all C^∞). $\nabla_h E$ is C^∞ by the same argument. The product of C^∞ functions is C^∞ . Hence $\tilde{h}$ is a C^∞ function of all named quantities.

Explicitly: $\frac{\partial \tilde{h}}{\partial \hat{h}} = I - \eta_r \cdot \frac{\partial}{\partial \hat{h}}[s(E(\hat{h}))\nabla_h E(\hat{h})]$ where $s = \sigma((E - \tau)/\beta_r)$ is smooth. This Jacobian is well-defined and computable by autograd. $\square$ $\square$

Remark 4.6 (Behaviour of Smooth Rejection). *When $E(\hat{h}) \ll \tau$: the gate $\sigma(\cdot) \rightarrow 0$ and $\tilde{h} \approx \hat{h}$ (no refinement). When $E(\hat{h}) \gg \tau$: gate $\rightarrow 1$ and $\tilde{h} \approx \hat{h} - \eta_r \nabla_h E$ (one gradient step toward low energy). This is a smooth interpolation between the two regimes, not a hard branch.*

Open Problem

Open: Whether the MLP verifier V learns to score genuinely *logically valid* hidden states (vs. states that are merely distributionally familiar) is unresolved. Resolving this requires: (a) an independent symbolic oracle to generate ground-truth validity labels; (b) a benchmark task where logical invalidity has clear consequences. Neither is provided in this paper.

5 Differentiable Logic Integration

The following is unchanged from v1 and is correct.

Definition 5.1 (Product-Logic Operators). *For $x, y \in [0, 1]$:*

$$x \wedge y := xy \quad (22)$$

$$x \vee y := x + y - xy \quad (23)$$

$$\neg x := 1 - x \quad (24)$$

Proven

Theorem 5.2 (Differentiability and Boolean Consistency). *The operators (22)–(24) are C^∞ on $\mathbb{R}$ with gradients $\partial(xy)/\partial x = y$, $\partial(x + y - xy)/\partial x = 1 - y$, and $\partial(1 - x)/\partial x = -1$. For $x, y \in \{0, 1\}$, they coincide with Boolean AND, OR, NOT.*

Proof. All three are polynomials of degree ≤ 2 , hence C^∞ . Boolean consistency follows by direct substitution. $\square$ $\square$

Remark 5.3 (Capacity Limitation of Logic Relaxation). *The reviewer correctly observes that applying product-logic operators to the output of the Panini mask does not constitute a “logic engine” in the formal sense. The relaxation faithfully computes soft AND/OR/NOT, but: (1) the inputs p_i are learned probabilities, not symbolic propositions; (2) the network has no mechanism to parse a formula from a state vector; (3) training on L_{total} provides no guarantee that the learned p_i represent any interpretable logical meaning. The v2 claim is restricted: product-logic operators allow structured penalty functions to be incorporated into the loss in a differentiable way. They do not provide formal reasoning.*

6 Stability and Convergence — Corrected Scope

6.1 Local Lyapunov Stability

Correction from v1

v1 error (reviewer §7.2): The v1 Lyapunov theorem required L_θ -smoothness of L_{total} globally, i.e., $\|\nabla^2 L_{\text{total}}\| \leq L_\theta$ for all θ . In a deep network with gating functions, sigmoid, GELU, and Stiefel projections, this constant is *unbounded globally*. The theorem as stated was inapplicable to real training.

v2 correction: The theorem is restricted to a *local* neighbourhood of a stationary point, where smoothness holds by compactness. The global convergence claim is retracted.

Definition 6.1 (Lyapunov Candidate). $V(\theta) := L_{\text{total}}(\theta) \geq 0$.

Proven

Theorem 6.2 (Local Lyapunov Descent). *Let θ^* be a stationary point of L_{total} and let $\mathcal{B}(\theta^*, r)$ be a ball on which L_{total} is L_θ -smooth (such a ball exists by continuity of $\nabla^2 L_{\text{total}}$ at θ^* under standard regularity conditions). For gradient descent with step size $\eta \leq 1/L_\theta$, as long as $\theta_t \in \mathcal{B}(\theta^*, r)$:*

$$V(\theta_{t+1}) \leq V(\theta_t) - \frac{\eta}{2} \|\nabla_\theta L_{\text{total}}(\theta_t)\|^2 \quad (25)$$

Proof. Local L_θ -smoothness gives $L_{\text{total}}(\theta_{t+1}) \leq L_{\text{total}}(\theta_t) + \langle \nabla L_{\text{total}}(\theta_t), \theta_{t+1} - \theta_t \rangle + \frac{L_\theta}{2} \|\theta_{t+1} - \theta_t\|^2$. With $\theta_{t+1} = \theta_t - \eta \nabla L_{\text{total}}(\theta_t)$: $= L_{\text{total}}(\theta_t) - \eta \|\nabla L_{\text{total}}(\theta_t)\|^2 + \frac{\eta^2 L_\theta}{2} \|\nabla L_{\text{total}}(\theta_t)\|^2 \leq L_{\text{total}}(\theta_t) - \frac{\eta}{2} \|\nabla L_{\text{total}}(\theta_t)\|^2$ for $\eta \leq 1/L_\theta$. $\square$ $\square$

Open Problem

Open: Global Convergence. The theorem above holds *locally*. For global descent, we would need either (a) global smoothness ($L_\theta < \infty$ everywhere), which fails

for sigmoid-gated deep networks; or (b) the Polyak-Łojasiewicz condition on L_{total} , which is unverified. Whether Adam (the practical optimiser) converges to a global minimum of L_{total} is unknown even for standard transformers.

Plausible — Unverified

Practical substitute for global Lyapunov: Gradient clipping (norm $\leq G$) combined with a cosine learning rate schedule effectively constrains the trajectory to a region where local smoothness holds throughout training. This is standard practice in large model training, but its interaction with the Stiefel constraint is not formally analysed here.

6.2 Riemannian Stability and fp16 Drift

Proven

Theorem 6.3 (Exact Orthogonality in Exact Arithmetic). *Under QR or Cayley retractions in exact real arithmetic, $S_{t+1}^\top S_{t+1} = I_K$ exactly at every step.*

Proof. By definition of a retraction $\text{Ret} : T_{\text{St}}(d, K) \rightarrow \text{St}(d, K)$. QR returns the Q factor with orthonormal columns; Cayley returns a matrix satisfying the Stiefel conditions via the Woodbury identity. $\square$ $\square$

Correction from v1

v1 error (reviewer §7.1): v1 presented the above as a practical guarantee. The reviewer correctly noted that **fp16 floating-point arithmetic accumulates rounding errors**, causing $S_t^\top S_t - I_K$ to drift. For $K = 100$, $d = 512$, this drift can become significant within hundreds of iterations.

v2 correction: The theorem holds in exact arithmetic only. In practice, the following engineering prescription is *required*:

Remark 6.4 (fp32 Stiefel Master Copy Protocol). *Standard mixed-precision training maintains fp32 master copies of parameters alongside fp16 working copies (standard since apex AMP). For the thread matrix S_t specifically:*

1. Store S_t as an fp32 buffer (cost: $K \cdot d \cdot 4$ bytes, negligible for $K = 8$, $d = 256$: ~ 8 KB).
2. Compute the Riemannian gradient and retraction **in fp32**.
3. Cast to fp16 only for the forward pass computation.
4. Every T_{orth} steps, apply **Gram-Schmidt re-orthogonalisation** to the fp32 copy: $S \leftarrow \text{QR}(S)[0]$. This costs $O(Kd^2)$ in fp32 and corrects accumulated drift.

Empirical target: $\|S_t^\top S_t - I_K\|_F < 10^{-4}$ after every T_{orth} steps (the prototype benchmarks this). This is an engineering prescription, not a proof.

Correction from v1

v1 error (reviewer §4.2 — Cayley for K=100): For $K = 100$, the Cayley retraction requires solving a 200×200 linear system at each step. In fp16, the

condition number of this system can be large, causing catastrophic cancellation and NaN propagation.

v2 correction: (a) Use QR retraction (not Cayley) for $K \geq 50$. The QR retraction costs $O(Kd^2)$ but is numerically stable (Householder reflections are backward-stable). (b) Store and compute the retraction in fp32 regardless of the training precision (as per Remark 6.4). (c) For $K = 1000$ (hypothetical Sahasrāvdhāna), the 2000×2000 Cayley solve is **not recommended in any precision**. The architecture does not support $K = 1000$ as specified.

6.3 Meta-Controller Stability (unchanged from v1)

Plausible — Unverified

The mirror-descent argument for controller stability is retained but downgraded from PROVEN to PLAUSIBLE because the global Lipschitz assumption it relies on has the same issue as Theorem 6.2: it holds locally but not globally.

7 Hardware Efficiency and Realistic Scale Analysis

7.1 FLOPs Accounting — Corrected

Correction from v1

v1 ambiguity (reviewer §3): The HBTA complexity proof was ambiguous about whether the top-down broadcast concatenates or sums level representations. If concatenation were used, leaf dimension becomes Ld and downstream cost becomes $O(NLd^2) = O(Nd^2 \log N)$, not $O(Nd \log N)$.

v2 correction: The broadcast is a gated sum (see Definition 2.4), maintaining leaf dimension d . The proof below is now unambiguous.

Proven

Theorem 7.1 (Total FLOPs: $O(Nd \log N)$). *Under fixed K, d, W, d_v , the full NSK forward pass costs $F(N) = O(Nd \log N)$ FLOPs.*

Proof. HBTA: At each of $L = \lceil \log_2 N \rceil$ levels: (a) Windowed attention over $2W + 1$ nodes per position: $N \cdot O(Wd^2)$; (b) Top-down gated sum: $N \cdot L \cdot O(d^2) = N \cdot O(d^2 \log N)$. Total HBTA: $O(Nd^2 \log N)$. For fixed d : $O(N \log N)$.

OTM Cayley ($K \leq 50$): $O(K^2d)$, constant in N .

OTM QR ($K > 50$): $O(Kd^2)$, constant in N .

Panini bottleneck: $O(Ndd') \leq O(Nd^2)$, written as $O(Nd)$ for fixed d .

Nyaya MLP: $O(Ndd_v)$, fixed for $d_v = O(d)$.

Total: Dominated by HBTA: $O(Nd^2 \log N)$. For fixed d : $\mathbf{F(N)} = \mathbf{O(N \log N)}$.

Caveat: The $O(Nd^2 \log N)$ count with explicit d is larger than the $O(N^2d)$ transformer for small N when $d^2 \log N > Nd$, i.e., when $d \log N > N$. For $d = 256$, $N = 4096$: $256 \cdot 12 = 3072 < 4096$, so HBTA is faster. For $d = 1024$, $N = 1024$: $1024 \cdot 10 = 10240 > 1024$, so HBTA is *slower* than attention for short sequences. This is expected: the advantage only materialises for $N \gg d \log N$. $\square$ $\square$

7.2 Realistic Hardware Feasibility

Correction from v1

v1 error (reviewer §4.1): $d = 256$ is unrealistic for modern sequence modelling. The v1 table was restricted to a toy scale.

v2 correction: We provide an extended table across realistic embedding dimensions and explicitly acknowledge where 16 GB VRAM is exceeded.

Table 1: Estimated VRAM for NSK mixed-precision training (fp16 activations, fp32 parameters). $K = 8$ (Ashta), $B = 16$, $T_{\text{orth}} = 100$. Shata ($K = 100$) adds $\approx 4\times$ OTM cost (negligible vs. activations).

d	N	Activations	Params	Optim state	Total
256	4096	0.5 GB	0.1 GB	0.2 GB	0.8 GB
512	4096	2.0 GB	0.4 GB	0.8 GB	3.2 GB
768	4096	4.5 GB	0.9 GB	1.8 GB	7.2 GB
1024	4096	8.0 GB	1.6 GB	3.2 GB	12.8 GB
1024	8192	16.0 GB	1.6 GB	3.2 GB	20.8 GB [†]

[†] Exceeds 16 GB; requires gradient checkpointing or batch size reduction.

Proven

Proposition 7.2 (16 GB Feasibility Bound). *For $d \leq 1024$, $N \leq 4096$, $B = 16$ (fp16 activations, fp32 params), total VRAM ≤ 12.8 GB < 16 GB. For $d = 1024$, $N = 8192$: total ≈ 20.8 GB; gradient checkpointing reduces activation memory by $\approx 5\times$ at $\sqrt{N}$ compute overhead, restoring feasibility.*

8 Gradient Flow: Corrected Global Analysis

Correction from v1

v1 error: Theorem 8.1 claimed $\nabla_{\theta} L_{\text{total}} \neq 0$ globally. This was wrong for two reasons established above: (1) mask saturation can drive $\partial L_{\text{verify}} / \partial c_k \rightarrow 0$; (2) gate saturation with a vanishing sub-network Jacobian can drive $\partial L_{\text{task}} / \partial h_{\ell} \rightarrow 0$.

v2 correction: The theorem is restricted to the *unsaturated regime* and the failure modes are explicitly characterised.

Proven

Theorem 8.1 (Gradient Non-Vanishing in the Unsaturated Regime). *Define the unsaturated regime as the set of parameters satisfying:*

- (U1) $g_{\ell}(x) \in [\epsilon, 1 - \epsilon]$ for some $\epsilon > 0$ (gates not saturated).
- (U2) $\|J_{f_{\ell}}\| \geq \nu > 0$ for some ν (sub-network Jacobians non-degenerate).
- (U3) $c_i \in [-C_{\text{max}}, C_{\text{max}}]$ for all i (constraint logits clipped).
- (U4) $V_j(h) \in [\epsilon, 1 - \epsilon]$ for all j (verifier not saturated).

Under (U1)–(U4), $\nabla_{\theta} L_{\text{total}} \neq 0$ for non-degenerate input distributions.

Proof. Under (U1): $\|\partial h_{\ell}/\partial h_0\| \geq \prod_k (1 - g_k) \epsilon^L > 0$. Under (U3): $(1 - \sigma(c_i)) \geq 1 - \sigma(C_{\text{max}}) > 0$. Under (U4): $(1 - V_j(h)) \geq \epsilon > 0$. Each component gradient is therefore bounded away from zero under its respective unsaturation condition, and their sum in L_{total} is non-zero. $\square$ $\square$

Table 2: Summary of gradient failure modes and prescribed mitigations.

Module	Failure condition	Mitigation
Pingala gate	$g_{\ell} \rightarrow 1$ and $J_{f_{\ell}} \rightarrow 0$	Init $b_{\ell} = 0$; LN before f_{ℓ} ; grad clip
Panini mask	$c_i \rightarrow +\infty$	Clip to $[-5, 5]$; entropy reg. L_{ent}
Nyaya verifier	$V_j \rightarrow 1$	Contrastive negatives; verifier noise
Stiefel (fp16)	fp16 drift	fp32 master copy; Gram-Schmidt every T_{orth}
Lyapunov bound	L_{θ} unbounded globally	Grad clip; cosine LR; local analysis only

9 Comparison with Existing Architectures

Table 3: Corrected architectural comparison. All NSK claims labelled.

Property	Transformer	Mamba	RWKV	Mem. Net.	nsk v2
Time (train)	$O(N^2 d)$	$O(Nd)$	$O(Nd)$	$O(NMd)$	$O(Nd^2 \log N)$
Memory	$O(N^2)$	$O(rd)$	$O(rd)$	$O(Md)$	$O(Nd)$
Thread isolation	–	–	–	Partial	Full [proven]
Plausibility verifier	–	–	–	–	MLP [plausible]
Grad-death mitigated	Yes	Yes	Yes	Yes	Yes [proven, local]
Lyapunov stable	–	–	–	–	Locally [proven]
Faster than $O(N^2)$	–	Yes	Yes	Partial	Yes [proven]
Faster than Mamba	N/A	N/A	N/A	N/A	No [proven]
Benchmark performance	SOTA	Competitive	Competitive	Limited	Unknown [open]

Mamba Asymptotic Dominance. NSK costs $O(Nd^2 \log N)$ vs. Mamba’s $O(Nd)$. For fixed d , NSK is $O(d \log N)$ times slower per token than Mamba. This is a **proven** disadvantage in raw throughput. NSK’s potential advantages are structural (thread isolation, hierarchical memory with $O(\log N)$ retrieval, constraint masking), and whether these yield better task performance is **empirically open**.

Open Problem

Open: Tree Boundary Artifacts. The reviewer correctly notes that HBTA enforces a binary-tree structure over the token sequence. For tasks requiring *arbitrary-range exact contiguous matching* (e.g., bracket-depth parity, code structure where a brace on token 1 must pair with token N), tree boundaries may break the signal entirely. HBTA’s window is local at each level; cross-tree attention is not available. Whether learned tree structures (e.g., latent tree grammars) could mitigate this is an open research question. Fixed binary trees are a concrete limitation.

10 Complete Open Problems and Limitation Inventory

10.1 Proven Results (Retained from v1, Scope Corrected)

1. HBTA retrieval $O(\log N)$ dot products (Theorem 2.5).
2. Orthogonality $S^\top S = I_K$ in *exact arithmetic* (Theorem 6.3).
3. Gated update differentiable with mitigation bound (Theorem 2.2).
4. Smooth rejection differentiable (Theorem 4.5).
5. Constraint masking C^∞ with saturation characterised (Theorem 3.4).
6. MLP verifier gradient non-zero in unsaturated regime (Proposition 4.2).
7. Product-logic operators C^∞ and Boolean-consistent (Theorem 5.2).
8. Local Lyapunov descent (Theorem 6.2).
9. Linear bottleneck = PCA (Proposition 3.2).
10. $O(Nd^2 \log N)$ total FLOPs, $O(N \log N)$ for fixed d (Theorem 7.1).
11. 16 GB VRAM feasible for $d \leq 1024$, $N \leq 4096$ (Proposition 7.2).
12. Gradient non-vanishing in the unsaturated regime (Theorem 8.1).

10.2 Plausible but Unverified

Plausible — Unverified

1. **Compute gating efficiency:** predicted $\bar{g} \approx 0.2\text{--}0.5$ under L_{compute} penalty — unverified on any benchmark.
2. **Semantic collapse reduction:** the mutual information argument assumes Gaussian tokens; real text distributions are unknown.
3. **Practical Stiefel stability:** fp32 master copy plus periodic Gram-Schmidt is standard in Riemannian ML, but $\|S^\top S - I\|_F < 10^{-4}$ over 10^5 training steps has not been measured for this architecture.
4. **Verifier plausibility learning:** the MLP verifier may learn a meaningful plausibility score or may collapse to a constant. Unknown.
5. **Controller stability:** the mirror-descent argument is sound under local Lipschitz conditions, but these conditions are unverified globally.

10.3 Open Problems (Extended from v1)

Open Problem

1. **Global convergence:** local Lyapunov holds; global does not. Loss landscape geometry of L_{total} is unknown.
2. **Nonlinear bottleneck:** whether MLP autoencoders converge to PCA or better is unknown for this architecture.

3. **Distance concentration in hierarchical routing:** greedy root-level mis-routing is unrecoverable in the current design. Beam search, temperature annealing, and ensemble routing are proposed but unimplemented.
4. **Tree boundary artifacts:** HBTA cannot attend across tree boundaries, potentially destroying signals for tasks requiring exact long-range matching.
5. **Empirical performance:** all complexity comparisons are theoretical. No benchmarks exist.
6. **Hyperparameter sensitivity:** $\alpha, \beta, \gamma, \delta$ in L_{total} , C_{max} , T_{orth} , τ , β_r have no principled derivation.
7. **Cayley for $K > 50$:** the QR retraction is prescribed but its numerical properties for $K = 100$ over long training runs are uncharacterised.
8. **Memory consolidation:** the $M^W \rightarrow M^E \rightarrow M^S$ consolidation policy is described but not formalised.
9. **AGI/ASI:** not claimed, not relevant. The system is a specialised differentiable architecture with specific, bounded properties.

11 Response to Reviewer: Point-by-Point

12 Conclusion

Peer review identified ten substantive mathematical errors in v1. All were accepted. The revisions are:

Corrected (no longer false): (1) Gradient death immunity $\rightarrow$ mitigation bound with explicit counter-example. (2) Nonlinear PCA claim $\rightarrow$ restricted to linear case only. (3) Non-differentiable Heaviside $\rightarrow$ smooth sigmoid gate. (4) “Non-degenerate” mask gradient $\rightarrow$ saturation characterised with prescribed mitigations. (5) Single linear verifier $\rightarrow$ two-layer MLP with restricted claims. (6) fp16 Stiefel purity $\rightarrow$ fp32 master copy + periodic Gram-Schmidt. (7) Global Lyapunov $\rightarrow$ local Lyapunov. (8) Ambiguous dimension-inflating broadcast $\rightarrow$ explicit gated-sum architecture.

Added as open problems: (9) Distance concentration and hierarchical routing fragility. (10) Mamba asymptotic dominance acknowledged.

After corrections, the NSK architecture has:

- **12 proven results**, all with corrected scope.
- **5 plausible-unverified claims**, labelled explicitly.
- **9 open problems**, including 3 new ones from this review.
- **0 claims we believe to be false**.

The architecture remains theoretically motivated and implementable. Whether it achieves competitive empirical performance is the central open question, answerable only by building and benchmarking it.

Table 4: Point-by-point response to the peer review. Every criticism accepted or rebutted with a specific reference to the correction.

Reviewer claim	Valid?	Action in v2
Gradient death immunity false	Yes	Replaced with mitigation bound + counter-example (§2.1)
Nonlinear bottleneck $\geq$ PCA false	Yes	Restricted to linear case; nonlinear labelled Open (§3.1)
Soft rejection Heaviside non-differentiable	Yes	Replaced with smooth sigmoid gate (§4.2)
Mask gradient $\rightarrow 0$ as $c_i \rightarrow \infty$	Yes	Saturation characterised; mitigations prescribed (§3.2)
Verifier gradient collapse at high confidence	Yes	Acknowledged; contrastive training prescribed (§4)
Dimension inflation $O(Nd^2 \log N)$	Partial	Sum architecture clarified; complexity with explicit d corrected (§7.1)
$d = 256$ unrealistic	Yes (practical)	Table extended to $d = 1024$ (§7.2)
Cayley ill-conditioned for $K \geq 100$ in fp16	Yes	QR prescribed for $K > 50$; fp32 master copy required (§6.2)
Distance concentration / routing fragility	Yes	Added as Open Problem with proposed mitigations (§2.2)
Linear verifier cannot model non-linear logic	Yes	Upgraded to 2-layer MLP; claims restricted to plausibility scoring (§4)
fp16 Riemannian drift	Yes	fp32 master copy + Gram-Schmidt protocol prescribed (§6.2)
Global Lipschitz L_θ unbounded	Yes	Lyapunov narrowed to local regime (§6.1)
Tree boundary artifacts for exact matching	Yes	Added as Open Problem (§9)
Mamba asymptotically faster	Yes	Explicitly stated in comparison table (§9)

References

- [1] Vaswani, A. et al. (2017). Attention is all you need. *NeurIPS*.
- [2] Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*.
- [3] Peng, B. et al. (2023). RWKV: Reinventing RNNs for the transformer era. *EMNLP 2023*.
- [4] Graves, A., Wayne, G., Danihelka, I. (2014). Neural Turing machines. *arXiv:1410.5401*.
- [5] Weston, J., Chopra, S., Bordes, A. (2015). Memory networks. *ICLR*.
- [6] Absil, P.A., Mahony, R., Sepulchre, R. (2008). *Optimization algorithms on matrix manifolds*. Princeton UP.
- [7] Bengio, E., Bacon, P.L., Pineau, J., Precup, D. (2015). Conditional computation in neural networks. *arXiv:1511.06297*.
- [8] Eckart, C., Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218.
- [9] Gödel, K. (1932). Zum intuitionistischen Aussagenkalkül. *Anzeiger Akademie Wien*, 69:65–66.
- [10] Dao, T. et al. (2022). FlashAttention. *NeurIPS 2022*.
- [11] Hochreiter, S., Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- [12] Indyk, P., Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. *STOC*.
- [13] Tishby, N., Pereira, F.C., Bialek, W. (2000). The information bottleneck method. *arXiv:physics/0004057*.
- [14] Panini (c. 4th c. BCE). *Ashtadhyayi*. Tr. Katre (1989).
- [15] Pingala (c. 3rd c. BCE). *Chandahshastra*.
- [16] Gautama (c. 2nd c. BCE). *Nyayasutra*. Tr. Jha (1913).